2 - Gradient-Boosting Decision Tree

1. Core Methodology

GBDT，全称为 Gradient Boosting Decision Tree，顾名思义，分为两部分，Gradient Boosting 和 Decision Tree，具体来说，就是使用**决策树**为基学习器的**梯度提升算法**，因此分别了解下这两部分的原理就可以很好的学习 GBDT 的思想。

2. Decision Tree

由于在 GBDT 中，每次迭代都需要用到**负梯度**，求**负梯度**就需要用到连续值，因此 GBDT 用的决策树都是 CART（classification and regression tree）回归树，而不是分类树，CART 回归树一般使用的节点分裂准则为平均平方误差 MSE，即在节点分裂时穷举每一个特征的每一个阈值，来寻找最优切分特征和最优切分点，衡量的方法是平方误差最小化。

3. Gradient Boosting

3.1 Boosting

Boosting，就是使用**前向分布算法**（Forward Stagewise Algorithm）来训练**加法**模型，加法模型表示如下：

$$f_M(x) = \sum_{i=1}^{M} T(x, \Theta)$$

这里 $f_M(x)$ 表示的就是经过 M 次迭代后最终得到的模型，而 $T(x, \Theta)$ 代表的就是单颗 CART 回归树，这里的 Θ 代表树的参数，这个模型的意义就是每次训练一个弱学习器 $T(x, \Theta)$，经过 M 次之后，将他们加总起来得到一个强的学习器 $f_M(x)$。

训练目标是使得最后的预测误差尽可能的小，但是预测集事先是不知道的，则只能使得训练集的训练误差最小，那么如何使得这种加法模型的训练误差最小？

答案是：**前向分步算法。**

3.2 Forward Stagewise Algorithm

在每次训练模型时，都拟合上一个模型预测值与真实值之间的差值 $y - \hat{y}$，也就是残差，然后再把这个模型加到原来的模型中，这样就可以使得更新后的模型的预测误差更小。

4. Boosting Tree

当 Boosting 使用的基学习器是树模型时，这种方法也被称为提升树（boosting tree），具体步骤如下：

1. 初始化 $f_0(x)=0$。
2. 对于 $m=1,2,\cdots,M$:
    1. 计算残差：$r_{mi}=y_i-f_{m-1}(x), \quad i=1,2,\cdots,N$
    2. 以残差 r_{mi} 为预测值，训练一个回归树 $T_m(x,\Theta)$
    3. 更新 $f_m(x)=f_{m-1}(x)+T_m(x,\Theta)$
3. 经过 M 次迭代后最终得到的提升树模型：

$$f_M(x)=\sum_{i=1}^{M}T(x,\Theta)$$

提升树每次迭代都拟合上一次模型中没有能够学习到的东西，也就是残差，使得提升树比一般决策树更强，但是，这种拟合残差的方式也会存在一定的缺陷（只是拟合负梯度的一种特例，即使用 MSE 损失函数），因此，基于提升树的改进算法 —— **GBDT** 就诞生了。

GBDT

从上述提升树算法中，可以发现，Boosting 的核心思想就是在每次训练模型时，都用新的模型来弥补之前模型的不足，使得模型越来越好，这实际上就是一个优化问题，提升树每次迭代的时候优化的就是残差，那有没有存在更好的优化方式呢？当然有，就是将这个模型的“不足”用**损失函数**的方式来表示，在每次迭代时优化这个损失函数即可！

损失函数可以用 $L(y,f(x))$，这里先用一个通用的形式来表示，具体的损失函数可以是各种各样的，比如回归问题的 MSE，MAE，分类问题的交叉熵等，然后我们的目标就是使得这个损失函数最小。

根据凸优化的理论，梯度的方向是函数值增长最快的方向，那么负梯度的方向自然就是函数值下降最快的方向，所以根据负梯度就可以找到函数值最小的地方。

那么，就可以通过拟合负梯度的形式来优化损失函数，也就是说，**在每次训练一个新模型的时候，都拟合上一个模型的损失函数的负梯度值，然后把这个新模型加到原来的模型中，这样必然使得更新后的模型的损失函数更小**，这个就是 Gradient Boosting 了，具体步骤如下：

1. 初始化 $f_0(x)=0$，并选取损失函数：$L(y,f(x))$
2. 对于 $m=1,\cdots,M$
    1. 计算负梯度：

$$-g_m(x_i)=-\left[\frac{\partial L(y,f(x_i))}{\partial f(x_i)}\right]_{f(x)=f_{m-1}(x)}$$

    2. 以负梯度 $-g_m(x_i)$ 为预测值，训练一个回归树 $T_m(x,\Theta)$
    3. 更新 $f_m(x)=f_{m-1}(x)+\rho T_m(x,\Theta)$
3. 经过 M 次迭代后得到最终模型：

$$f_M(x)=\sum_{i=1}^{M}T(x,\Theta)$$

这个 ρ 也被称为**学习率**（learning rate），为什么要加这个学习率呢？如果将学习率设为1，就等于一下子学习全部的不足，会导致过拟合。因此，可以通过设置较小的学习率和较大的迭代次数以得到一个较优的模型，当然，这是以计算效率为代价的。

Why negative gradient?

残差只是损失函数负梯度的一种特例，而采用负梯度更加通用，可以使用不同的损失函数。

考虑如下损失函数：

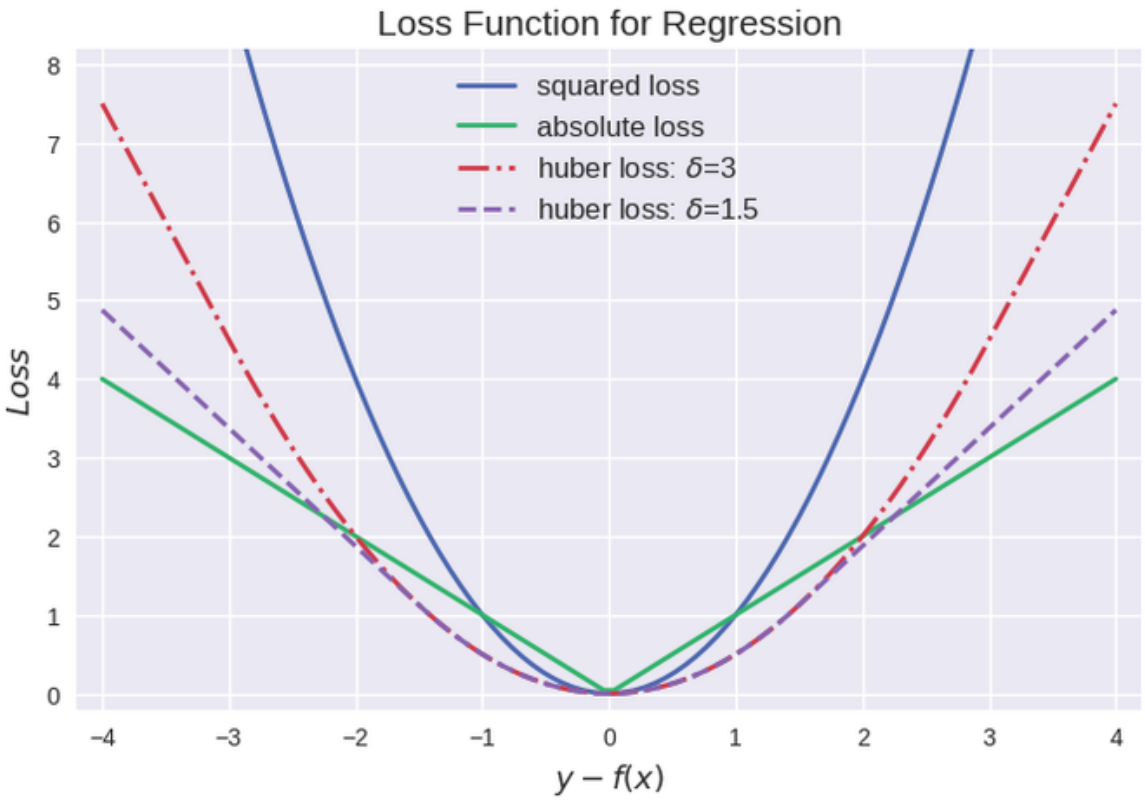
$$L(y,f(x))=\frac{(y-f(x))^2}{2}$$

即平均平方误差 MSE，它的负梯度就是：

$$-\left[\frac{\partial L(y,f(x))}{\partial f(x)}\right]=y-f(x)$$

即为残差。

也就是说，残差等价于当损失函数采用 MSE 时的负梯度，而损失函数采用 MSE 其实会有一些缺点，比如对异常值敏感，所以在实际问题中有可能采用 MAE 或者更为折中的 Huber 损失函数以避免这一缺陷，如图：



并且呢，这些损失函数的负梯度其实也可以看作是残差的一个近似，如图：

Setting	Loss Function	$-\partial L(y_i, f(x_i))/\partial f(x_i)$
Regression	$\frac{1}{2}[y_i - f(x_i)]^2$	$y_i - f(x_i)$
Regression	$\lvert y_i - f(x_i)\rvert$	$\text{sign}[y_i - f(x_i)]$
Regression	Huber	$y_i - f(x_i)$ for $\lvert y_i - f(x_i)\rvert \le \delta_m$ $\delta_m \text{sign}[y_i - f(x_i)]$ for $\lvert y_i - f(x_i)\rvert > \delta_m$ where $\delta_m = \alpha\text{th-quantile}\{\lvert y_i - f(x_i)\rvert\}$
Classification	Deviance	kth component: $I(y_i = \mathcal{G}_k) - p_k(x_i)$

Another view

为什么要使用负梯度可以这么理解:

$L(y, f(x))$ 中看似有两个变量，其实自变量只有 $f(x)$，因为 y 是已知且固定的，$GBDT$ 实际上最小化 L 用的还是最速下降法，即 $f'(x) = f(x) + \alpha\nabla L$，但是问题是 L 无法对 $f(x)$ 求梯度（如果使用回归树），所以只能去再训练一个决策树 $h(x)$，用它来预测当前输入 x 的情况下梯度是多少，最后得到的强分类器实际上求的就是极小值点。

这也是为什么 $GBDT$ 中用的弱回归器一般都是决策树，如果用的是线性回归的话, L 对 $f(x)$ 的梯度可以直接计算得出，那么也就不需要再训练一个回归器来预测梯度了（换句

话说用的回归器必须是离散的，无法求梯度的）